Panadería “Al Paso”: HTML, CSS y JavaScript

Informe del proyecto web

Institución: [completar]

Carrera: [completar]

Trabajo práctico: [completar]

Materia: Aplicaciones Web 1

Profesor: Andres Senn

Alumno: Francisco Valdomero

Fecha: [completar]

Índice

1. Resumen **3**

2. Estructura final del proyecto **3**

3. Etapas del proyecto **4**

Etapa 1 — Análisis del proyecto HTML 4
Etapa 2 — Corrección del proyecto HTML 4
Etapa 3 — Primera hoja de estilos 5
Etapa 4 — Rediseño visual de la hoja de estilos 5
Etapa 5 — Ajustes de catálogo y ubicación 5
Etapa 6 — Revisiones y mejoras posteriores 6
Etapa 7 — Verificación 6
Etapa 8 — Integración con JavaScript 7

4. Cambios por archivo **7**

5. Sistema de estilos: decisiones de diseño **8**

6. Nomenclatura de clases **9**

7. Semántica y accesibilidad **11**

8. Cumplimiento de los criterios de evaluación **12**

9. Integración con JavaScript **13**

9.1 Arquitectura modular 13
9.2 Datos del catálogo en un módulo independiente 14
9.3 Renderizado del catálogo 14
9.4 Carrito dinámico 14
9.5 Vistas del proceso de compra 15
9.6 Cuenta de cliente y pedidos 15
9.7 Eventos utilizados 15
9.8 Tipos de funciones utilizados 16
9.9 Comentarios y convenciones 16
9.10 Verificación 17
9.11 Cambios respecto de la propuesta 17

10. Pendientes y recomendaciones **18**

11. Conclusiones **18**

Informe de cambios — Panadería Al Paso

Proyecto web de catálogo y ventas online · Octubre de 2026

1. Resumen

Panadería Al Paso es un proyecto web que busca digitalizar el catálogo de una panadería de barrio, permitir el registro de clientes y realizar la compra online de sus productos. Este informe documenta el desarrollo en dos grandes partes. En la primera se revisó el proyecto original (seis páginas HTML sin estilos), se corrigieron errores de estructura, rutas, formularios y accesibilidad, y se creó la hoja de estilos `css/styles.css` con un diseño responsive. En la segunda se integró JavaScript: el catálogo se dibuja desde un módulo de datos, el carrito es dinámico, el proceso de compra tiene todas sus vistas (carrito, entrega, pago y confirmación) y las cuentas de cliente funcionan en modo simulado.

El código JavaScript está organizado en módulos (`import` / `export`) dentro de la carpeta `js`, con comentarios en cada parte. Todos los estilos se encuentran en `css/styles.css` y los archivos HTML no contienen estilos propios. Las imágenes están en `recursos/imagenes`. Como el sitio usa módulos de JavaScript, debe abrirse con un servidor local (por ejemplo, la extensión Live Server de VS Code) y no con doble clic sobre el archivo. La sección 9 detalla la integración con JavaScript y los cambios respecto de la propuesta original.

2. Estructura final del proyecto

El proyecto se organiza en la carpeta `src`. La página inicial, `index.html`, se encuentra en su raíz, y los recursos están separados en directorios: `css` para la hoja de estilos, `js` para el código JavaScript y `recursos/imagenes` para las imágenes.

```
src/
├── index.html
├── comoFunciona.html
├── catalogo.html
├── carrito.html
├── entrega.html
├── pago.html
├── confirmacion.html
├── cuenta.html
├── ubicacion.html
├── README.md
├── css/
│   └── styles.css
├── js/
│   ├── principal.js
│   ├── aviso-archivo.js
│   ├── datos/          (catálogo, pagos, entrega)
│   ├── modulos/        (lógica y utilidades)
│   ├── componentes/    (resumen)
│   └── vistas/         (una por página)
└── recursos/
    └── imagenes/
```

3. Etapas del proyecto

El trabajo se desarrolló en ocho etapas. Las siete primeras corresponden a HTML y CSS y la octava a la integración con JavaScript, que se detalla en la sección 9.

Etapa 1 — Análisis del proyecto HTML

Se analizaron los seis archivos y se detectó lo siguiente:

- Rutas de imágenes con barra invertida (`recursos\imagenes\chipa.png`), que fallan en servidores Linux.
- Formularios del catálogo sin identificar el producto enviado.
- Formularios de cuenta sin forma de distinguir el registro del inicio de sesión.
- Enlaces vacíos (`href="#"`) en las redes sociales.
- Uso de `<cite>` para nombres de personas en los testimonios.
- Falta del enlace «Saltar al contenido», de `aria-current` en el menú y de `meta description`.
- Texto alternativo de la imagen principal que no coincidía con el nombre del archivo.

Etapa 2 — Corrección del proyecto HTML

Rutas y enlaces

- Todas las rutas de imágenes usan `/` en lugar de `\`.
- Cada página vincula `css/styles.css`.
- El teléfono pasó a ser un enlace `tel:` y se agregó un enlace de WhatsApp (`wa.me`).
- Las redes sociales tienen URL provisorias con `target="_blank"` y `rel="noopener noreferrer"`.

Accesibilidad y SEO

- Enlace «Saltar al contenido» al inicio de cada página.
- `aria-current="page"` en el ítem activo del menú.
- `meta description` y `theme-color` en todas las páginas.
- Nuevo texto alternativo de la imagen principal: «Canasto con chipas, medialunas y facturas recién horneadas».

Catálogo

- Cada formulario envía el código del producto (campo oculto `producto`) y la cantidad.
- El código se separó del título del producto.
- El precio usa `<data value="...">`.
- Las imágenes tienen `loading="lazy"`.
- Los testimonios usan `figure`, `blockquote` y `figcaption`, sin `<cite>`.

Formularios de cuenta

- Campo oculto `accion` (registro o login).
- Atributos `autocomplete` en cada campo.

- Teléfono con `inputmode` y `pattern`.

Contenido

- Se quitó la numeración «01, 02, 03» escrita en «Cómo funciona»; ahora la genera el CSS con contadores.

Etapa 3 — Primera hoja de estilos

Se creó `css/styles.css` con enfoque mobile-first, variables de color y tipografía, foco visible, `prefers-reduced-motion` y estilos de impresión. Esta primera versión incluyó encabezado fijo, grillas adaptables, tabla del carrito convertida en tarjetas en celular y validación visual de formularios.

Al probar las páginas, los estilos no se aplicaban porque el archivo no estaba en la ruta `css/styles.css` que enlazan los HTML. Al ubicarlo correctamente, el problema se resolvió. Esta experiencia muestra la importancia de que las rutas coincidan exactamente, incluidas las mayúsculas y minúsculas.

Etapa 4 — Rediseño visual de la hoja de estilos

Se reescribió `styles.css` para lograr el aspecto definido en el diseño de referencia del proyecto. Los criterios de diseño fueron los siguientes:

- **Paleta:** fondo crema, cacao oscuro, terracota como acento, ámbar para detalles y azul para la insignia del carrito.
- **Tipografías:** Fraunces (títulos y citas), Karla (texto) y Space Mono (etiquetas, códigos y precios), cargadas desde Google Fonts.
- **Encabezado:** logo con ✺, menú centrado con subrayado terracota en la página activa, ícono de carrito circular con insignia.
- **Inicio:** título con «al paso.» en cursiva, botones de píldora y tarjeta tipo ticket sobre la foto con el sello «Listo». Se eliminó la sección «El producto de la semana», cuyo contenido pasó al ticket.
- **Catálogo:** filtros en píldoras, tarjetas con etiqueta de código sobre la imagen, precio en tipografía monoespaciada y botón «+» circular. Franja clara para los testimonios.
- **Cuenta:** tarjeta con pestañas «Crear cuenta» e «Iniciar sesión» que cambian de formulario solo con CSS (`:target` y `:has()`).
- **Pie de página:** borde superior festoneado, cuatro columnas y leyenda del sitio.
- **Cambios de HTML asociados:** nueva estructura de la imagen principal, de la página de cuenta y del pie; el enlace «Iniciar sesión» del menú ahora apunta a `cuenta.html#login`.

Etapa 5 — Ajustes de catálogo y ubicación

- **Catálogo en filas de 4:** los productos estaban agrupados por categoría, cada una con su propia grilla (2, 2, 3 y 1 productos). Se unificaron en una sola grilla de 8: 4 columnas desde 1000 px, 3 desde 760 px, 2 desde 480 px y 1 en pantallas menores. Los filtros conducen al primer producto de cada categoría mediante `id`.
- **Ubicación:** se reemplazó la imagen `local.svg` por un panel de texto y, después, por un mapa de Google Maps incrustado (`<iframe>` con `title` descriptivo, carga diferida y enlace «Cómo llegar»). El mapa se busca por la dirección escrita, sin coordenadas.

Etapa 6 — Revisiones y mejoras posteriores

Menú y tabla

- Se agregó «Inicio» (`index.html`) como primer ítem del menú principal, con `aria-current` cuando corresponde. También se agregó al pie.
- La tabla del carrito se mantiene como tabla en escritorio. En pantallas menores de 680 px, cada fila pasa a ser una tarjeta con `display: flex`, y las celdas muestran su etiqueta mediante `data-label`.

Página de ubicación (ubicacion.html)

- Se envolvieron en `<address>` los datos de contacto: dirección y teléfono. El horario y los envíos quedan en un segundo `<dl>` porque no son datos de contacto.
- Se agregó `font-style: normal` a `address` para evitar la cursiva que aplican los navegadores por defecto.
- El orden de los datos cambió: dirección, teléfono, horario y envíos.

Etiquetas <p> sin función de párrafo

Se reemplazaron por `<div>` en los sobretítulos, los grupos de botones (`.acciones`), los precios y los campos de formulario (`.campo`). Los `<p>` que contienen texto real se conservaron.

Responsive

- El encabezado pasa a dos filas hasta 899 px.
- Por debajo de 480 px se centra y deja de ser fijo.
- Los enlaces del menú y el botón «+» tienen áreas táctiles mayores en móvil.
- El título de la tabla (`caption`) se corrigió en celular.

Etapa 7 — Verificación

Se verificó el funcionamiento del sitio con pruebas automáticas y con una revisión manual en el navegador de escritorio. Los resultados se resumen en la Tabla 1.

Tabla 1. *Verificaciones realizadas*

Prueba	Alcance	Resultado
Desbordes horizontales (Chromium)	10 anchos de 320 a 1920 px × 6 páginas	0 problemas
Accesibilidad automática (axe-core, WCAG 2 A/AA)	6 páginas, a 375 px y 1280 px	1 falla inicial (contraste del sello «Listo»), corregida; resultado final: 0 violaciones
Estructura del HTML	Apertura y cierre de etiquetas en los 6 archivos	Correcto
Rutas de recursos	Búsqueda de barras invertidas en las rutas	Ninguna encontrada

La auditoría automática no reemplaza una prueba con lector de pantalla, que queda como mejora pendiente.

Etapa 8 — Integración con JavaScript

Se incorporó el código JavaScript que da funcionamiento al sitio, con los siguientes resultados principales:

- El catálogo se dibuja completo desde JavaScript, a partir de un módulo de datos independiente, con filtros por categoría, buscador y orden.
- El carrito es dinámico: se guarda en el navegador, se actualiza en todas las páginas y permite cambiar cantidades, quitar productos y vaciarlo.
- Se crearon las vistas que faltaban del proceso de compra: entrega, pago y confirmación con ticket.
- Las cuentas de cliente funcionan en modo simulado, con historial de pedidos y opción de repetir un pedido.
- El código se organizó en módulos, con comentarios que explican cada parte.

4. Cambios por archivo

Tabla 2. *Resumen de cambios en cada archivo*

Archivo	Cambios principales
`index.html`	Imagen principal reestructurada con ticket de producto, «Inicio» en el menú, sección «El producto de la semana» eliminada, botones en `div`.
`comoFunciona.html`	Numeración generada por CSS y encabezado de página unificado.
`catalogo.html`	Etapa HTML/CSS: grilla única de 8 productos, `data` en los precios y testimonios con `figure`. Etapa JavaScript: el catálogo ya no está escrito en el HTML; un contenedor (`#catalogo-app`) lo recibe desde JavaScript.
`ubicacion.html`	`address` con dos `dl`, mapa incrustado y enlace «Cómo llegar».
`cuenta.html`	Pestañas con `:target`, campos en `div`, `autocomplete` y `pattern`. Etapa JavaScript: registro e inicio de sesión funcionales (simulados), panel con historial de pedidos y cierre de sesión.
`carrito.html`	Etapa HTML/CSS: tabla con `data-label` para la versión móvil. Etapa JavaScript: la tabla de ejemplo se reemplazó por el carrito real dibujado desde JavaScript, con barra de progreso de la compra.
`css/styles.css`	Archivo nuevo: sistema de diseño completo, responsive, con comentarios por sección. En la etapa JavaScript se agregaron los estilos de los componentes dinámicos.
`entrega.html`, `pago.html`, `confirmacion.html`	Páginas nuevas: pasos 2, 3 y 4 de la compra.
`index.html`, `comoFunciona.html`, `ubicacion.html`	Se agregaron los scripts y el atributo `data-pagina`; el precio del ticket de inicio se toma del módulo de datos.
`js/` (23 archivos)	Carpeta nueva con todo el código JavaScript (sección 9).
`README.md`	Archivo nuevo: instrucciones para abrir el sitio con un servidor local.

En los archivos HTML también se corrigieron las rutas y se agregaron el enlace al CSS, el enlace «Saltar al contenido», la `meta description`, el atributo `aria-current`, el nuevo encabezado con ícono de carrito y el nuevo pie de página.

5. Sistema de estilos: decisiones de diseño

El estilo de la página web lo define el archivo `styles.css`, un único archivo organizado en secciones comentadas y construido con enfoque mobile-first: los estilos base son los del celular y las media queries amplían el diseño para pantallas mayores.

Variables y tipografía

Los colores y las tres familias tipográficas están definidos como variables en `:root`, de modo que un cambio de paleta se hace en un solo lugar. Las tipografías son Fraunces (títulos y citas), Karla (texto) y Space Mono (etiquetas, códigos y precios), todas con alternativas de sistema.

Tabla 3. *Variables de color*

Variable	Valor	Uso
`--crema`	#f6eddc	Fondo general
`--crema-clara`	#fffaf0	Tarjetas y franja de testimonios
`--cacao`	#2b1b12	Texto principal y pie de página
`--terracota`	#a63f2a	Acento: botones, enlaces y página activa
`--ambar`	#e5a437	Detalles y títulos del pie
`--azul`	#3f7fb0	Insignia del carrito
`--suave`	#6a5848	Texto secundario
`--linea`	#e4d6bb	Bordes y líneas punteadas

Organización del archivo

Las secciones del archivo siguen el orden de lectura del sitio.

Tabla 4. *Secciones de la hoja de estilos*

Sección	Contenido
Base	Variables, reinicio, tipografía, foco visible y clase `.sr-only`
Encabezado	Menú, ícono de carrito e insignia
Botones	Botón principal y secundario
Encabezado de página	Sobretítulo, título y línea punteada
Inicio	Imagen principal, ticket de producto y sello
Cómo funciona	Pasos con contadores CSS
Catálogo	Filtros, grilla, tarjetas y testimonios
Formularios	Campos y estados de foco y de error
Carrito	Tabla y medios de pago

Sección	Contenido
Ubicación	Datos de contacto y mapa
Cuenta	Tarjeta con pestañas
Pie	Borde festoneado y columnas

Técnicas utilizadas

Se utilizó CSS Grid para las grillas de página y de productos; Flexbox para los menús y para la tabla del carrito en celular; `clamp()` para tipografía y márgenes fluidos; `aspect-ratio` para que las imágenes no se desplacen al cargar; contadores CSS para numerar los pasos; `:target` y `:has()` para las pestañas sin JavaScript; y un degradado radial para el borde festoneado del pie.

Puntos de corte (media queries)

Tabla 5. *Puntos de corte y cambios de diseño*

Ancho	Qué cambia
hasta 479 px	Encabezado centrado y sin posición fija
hasta 679 px	La tabla del carrito pasa a tarjetas con Flexbox
hasta 899 px	El menú pasa a una segunda fila
desde 480 px	Catálogo en 2 columnas
desde 600 px	Medios de pago en 2 columnas
desde 760 px	Catálogo en 3 columnas
desde 800 px	Pasos y testimonios en 3 columnas; pie en 4 columnas
desde 860 px	Inicio, ubicación y cuenta en 2 columnas
desde 960 px	Medios de pago en 4 columnas
desde 1000 px	Catálogo en 4 columnas

Además, hay reglas para `prefers-reduced-motion` (sin animaciones si el usuario lo solicita) y para impresión (se ocultan el encabezado, el pie y los formularios).

Estilos de la etapa JavaScript

Para los componentes dinámicos se agregó un bloque al final de `styles.css`: avisos temporales, barra de búsqueda y filtros del catálogo, controles de cantidad del carrito, barra de progreso de la compra, formularios de entrega y pago, resumen del pedido, ticket de confirmación y panel de cuenta. Los filtros del catálogo pasaron de ser enlaces a botones con `aria-pressed`, y se agregó la regla `[hidden]` para que el atributo oculte siempre los elementos. Los componentes nuevos mantienen la paleta, las tipografías y los puntos de corte ya definidos.

6. Nomenclatura de clases

Las clases CSS siguen un criterio común de nombres:

- **Idioma:** los nombres están en español, salvo algunas clases habituales (`.hero`, `.lead`, `.sr-only`).
- **Formato:** minúsculas separadas por guiones.
- **Jerarquía por prefijo:** las partes de un componente repiten su nombre como prefijo. Por ejemplo, `.producto` contiene a `.producto-imagen`, `.producto-cuerpo` y `.producto-pie`.
- **Modificadores:** variantes de un componente base, como `.boton` y `.secundario`, o `.sobretitulo` y `.sobretitulo-acento`.
- **Utilidades:** clases genéricas de apoyo, como `.sr-only` y `.saltar`.
- **Elementos estructurales:** el encabezado, el pie y los menús se estilizan con selectores de elemento.

Tabla 6. *Clases principales por componente*

Componente	Clases	Descripción
Encabezado	`.marca`, `.marca-icono`, `.enlace-cuenta`, `.carrito`, `.insignia`	Logo, enlace de cuenta y carrito con contador
Botones	`.boton`, `.secundario`, `.acciones`	Botón, variante y grupo de botones
Encabezado de página	`.encabezado-pagina`, `.sobretitulo`, `.sobretitulo-acento`	Título de cada página
Inicio	`.hero`, `.ventajas`, `.ticket`, `.ticket-codigo`, `.ticket-precio`, `.sello`	Presentación y producto destacado
Cómo funciona	`.pasos`	Tres pasos numerados
Catálogo	`.filtros`, `.productos`, `.producto`, `.producto-imagen`, `.producto-cuerpo`, `.producto-pie`, `.codigo`, `.precio`, `.mas`, `.banda`, `.testimonios`	Filtros, grilla de productos y testimonios
Carrito	`.tabla-wrap`, `.celda-producto`, `.medios`, `.nota`	Tabla y medios de pago
Ubicación	`.contacto`, `.datos`, `.mapa`	Datos y mapa
Cuenta	`.cuenta-grid`, `.cuenta-lado`, `.beneficios`, `.tarjeta-cuenta`, `.pestanas`, `.formulario`, `.campo`, `.ayuda`	Formularios con pestañas
Pie	`.pie-grid`, `.pie-marca`, `.pie-base`, `.pie-claro`	Pie de página
Barra del catálogo	`.catalogo-barra`, `.catalogo-controles`, `.campo-inline`, `.catalogo-estado`, `.estado-vacio`	Filtros, buscador, orden y mensajes
Carrito dinámico	`.cantidad-control`, `.cantidad-boton`, `.boton-texto`, `.acciones-carrito`	Controles de cantidad y acciones
Proceso de compra	`.progreso`, `.pasos-compra`, `.checkout-grid`, `.formulario-pedido`, `.opciones`, `.opcion`, `.resumen`, `.ticket-pedido`	Pasos, formularios y comprobante
Cuenta y avisos	`.panel-cuenta`, `.pedidos`, `.pedido`, `.mensaje-error`, `.aviso`	Historial de pedidos y mensajes

Componente	Clases	Descripción
Utilidades	`.sr-only`, `.saltar`	Texto solo para lectores de pantalla y enlace de salto

Limitaciones. No se aplicó una metodología formal como BEM. Algunos modificadores no llevan el prefijo de su componente, y varios selectores del menú dependen del texto del atributo `aria-label`. Estos puntos se contemplan como mejoras en la sección 10.

7. Semántica y accesibilidad

Las páginas pasaron la auditoría automática de axe-core (WCAG 2 A/AA) con 0 violaciones, tanto en su versión estática como en los estados dinámicos (carrito con productos, formulario de envío, pago, confirmación y panel de cuenta), a 360, 768 y 1280 px. La auditoría no reemplaza una prueba con lector de pantalla.

Tabla 7. *Recursos de semántica y accesibilidad aplicados*

Aspecto	Cómo se resolvió
Estructura de la página	`header`, `nav`, `main`, `section`, `article`, `figure` y `footer`; un solo `h1` por página y jerarquía de títulos sin saltos
Navegación	Cada `nav` con `aria-label` propio; `aria-current="page"` en la página activa; enlace «Saltar al contenido»
Secciones	`aria-labelledby` apuntando al título de cada sección
Formularios	Cada campo con su `label`, atributos `autocomplete` y validación nativa (`required`, `minlength`, `pattern`)
Imágenes	Texto alternativo en las imágenes con contenido; el sello decorativo está oculto con `aria-hidden`
Datos	`address` para los datos de contacto, `dl` para pares de etiqueta y valor, `data` para los precios
Tabla del carrito	`caption`, `thead`, `tfoot` y `scope` en los encabezados
Teclado y movimiento	Foco visible en todos los elementos interactivos y respeto de `prefers-reduced-motion`
Contraste	Se corrigió el color del sello «Listo», único punto detectado por la auditoría
Idioma	`lang="es-AR"` en todas las páginas
Contenido dinámico	Región `aria-live` para los avisos, estado `role="status"` en el catálogo y el carrito, y mensajes de error con `role="alert"`
Foco	Tras redibujar el carrito, el foco vuelve al control que el usuario estaba usando
Filtros y pasos	Botones con `aria-pressed` y barra de progreso con `aria-current="step"`

8. Cumplimiento de los criterios de evaluación

Tabla 8. *Estado de cada criterio*

Criterio	Estado	Observación
Páginas HTML del TP1	Cumple	Las seis páginas están creadas y enlazadas entre sí.
Página inicial `index.html`	Cumple	Presenta el negocio y conduce al catálogo.
`index.html` en la raíz	Cumple	Se ubica en la raíz de la carpeta del proyecto (`src`), no en una subcarpeta.
Recursos en directorios distintos	Cumple	`css/` para los estilos y `recursos/imagenes/` para las imágenes.
Etiquetado semántico y ARIA	Cumple	Ver sección 7.
Estilos CSS organizados	Cumple	Un archivo con secciones comentadas y variables (sección 5).
Responsividad con media queries	Cumple	Ver sección 5; verificado de 320 a 1920 px sin desbordes.
Nomenclatura de clases	Parcial	Hay un criterio general de nombres, pero no una convención formal (sección 6).

Requisitos de la etapa de integración con JavaScript:

Tabla 9. *Estado de los requisitos de JavaScript*

Requisito	Estado	Dónde está
Renderizado completo del catálogo vía JavaScript	Cumple	`js/vistas/catalogo.js`; el HTML solo tiene un contenedor.
Carrito dinámico	Cumple	`js/modulos/carrito.js` y `js/vistas/carrito.js`.
Datos del catálogo en un módulo independiente	Cumple	`js/datos/catalogo.js`.
Código organizado y modularizado	Cumple	23 archivos en 5 carpetas con módulos ES.
Eventos JavaScript	Cumple	Tabla 12 de la sección 9.
Vistas del proceso de compra completas	Cumple	`entrega.html`, `pago.html` y `confirmacion.html`, además de carrito y cuenta.
Distintos tipos de funciones	Cumple	Tabla 13 de la sección 9.
Comentarios en cada parte del código	Cumple	Cabecera en cada archivo y comentario en cada función o bloque.
Informe en PDF con cambios respecto de la propuesta	Cumple	Este documento, sección 9.11.

9. Integración con JavaScript

Esta sección describe lo desarrollado en la etapa de JavaScript: la arquitectura del código, cada funcionalidad, los eventos y funciones utilizados, la verificación realizada y los cambios respecto de la propuesta original.

9.1 Arquitectura modular

El código se escribió con módulos de JavaScript (`import` y `export`). Cada archivo tiene una única responsabilidad y declara de qué otros depende. Todas las páginas cargan un único punto de entrada, `js/principal.js`, que inicia lo común (encabezado y avisos) y descarga, mediante `import()` dinámico, solo la vista de la página actual, que se identifica con el atributo `data-pagina` del `body`.

Tabla 10. *Módulos de JavaScript y su responsabilidad*

Capa	Archivos	Responsabilidad
Entrada	`principal.js`, `aviso-archivo.js`	Arranque de cada página; aviso si el sitio se abre sin servidor.
Datos	`datos/catalogo.js`, `datos/pagos.js`, `datos/entrega.js`	Solo datos: productos, categorías, medios de pago, cuotas, franjas de envío y datos del local.
Almacenamiento	`modulos/almacenamiento.js`	Lectura y escritura segura en localStorage, con prefijo y manejo de errores.
Lógica de negocio	`modulos/productos.js`, `modulos/carrito.js`, `modulos/pedidos.js`, `modulos/sesion.js`	Búsqueda y filtrado, estado del carrito, proceso de compra y cuentas simuladas.
Utilidades	`modulos/plantillas.js`, `formato.js`, `dom.js`, `avisos.js`, `encabezado.js`	Plantillas HTML seguras, formato de precios y fechas, atajos del DOM, avisos y encabezado dinámico.
Componentes	`componentes/resumen.js`	Resumen del pedido reutilizado en entrega y pago.
Vistas	`vistas/inicio.js`, `catalogo.js`, `carrito.js`, `entrega.js`, `pago.js`, `confirmacion.js`, `cuenta.js`	Una por página: dibujan la interfaz y manejan sus eventos.

9.2 Datos del catálogo en un módulo independiente

Los productos y categorías están en `js/datos/catalogo.js`, un módulo que solo exporta datos y no contiene lógica ni toca el DOM. Las estructuras se congelan con `Object.freeze` para evitar modificaciones accidentales. Para cambiar un precio, agregar un producto o conectar una API en el futuro, solo hay que editar este archivo. Un producto tiene la siguiente forma:

```
{ codigo: '014',
  nombre: 'Chipas (x12)',
  categoria: 'chipas',
  precio: 4200,
  descripcion: 'Recién horneadas, con queso de verdad.',
  imagen: 'chipa.png',
  alt: 'Chipas recién horneadas' }
```

Los medios de pago, las cuotas y los datos de entrega siguen el mismo criterio en `datos/pagos.js` y `datos/entrega.js`, y se usan para dibujar tanto el carrito como los pasos de la compra, de modo que cada dato exista una sola vez.

9.3 Renderizado del catálogo

`catalogo.html` ya no contiene productos: solo un contenedor vacío (`#catalogo-app`) con un mensaje para quienes tengan JavaScript desactivado. La vista `vistas/catalogo.js` dibuja la barra de filtros, el buscador, el selector de orden y la grilla de tarjetas a partir del módulo de datos.

- **Filtros por categoría:** botones con `aria-pressed`. Un enlace como `catalogo.html#chipas` abre el catálogo con ese filtro aplicado.
- **Buscador:** ignora tildes y mayúsculas y espera 250 ms después de la última tecla antes de filtrar.
- **Orden:** por código, nombre o precio, ascendente o descendente.
- **Plantillas seguras:** el módulo `plantillas.js` ofrece la función etiqueta `html`, que escapa automáticamente todo valor interpolado y evita la inyección de HTML.
- **Estados:** mensaje cuando no hay resultados y un texto de estado («Mostrando 8 productos») anunciado a lectores de pantalla.

9.4 Carrito dinámico

- **Estado:** se guarda solo el código y la cantidad de cada producto. Los nombres y precios se toman siempre del catálogo, por lo que un valor alterado en el navegador no cambia el precio.
- **Persistencia:** se guarda en localStorage, por lo que sobrevive al recargar la página o cerrar el navegador.
- **Operaciones:** agregar, cambiar cantidad, quitar y vaciar, con un máximo de 20 unidades por producto.
- **Comunicación entre módulos:** cada cambio emite el evento personalizado `carrito:cambio`; el encabezado (insignia con la cantidad) y la tabla del carrito se redibujan al recibirlo.
- **Varias pestañas:** el evento `storage` sincroniza el carrito si se modifica desde otra pestaña.
- **Accesibilidad:** al redibujar la tabla, el foco vuelve al control que se estaba usando, y un texto `aria-live` anuncia el nuevo total.

9.5 Vistas del proceso de compra

La propuesta planteaba un proceso en tres pasos (elegir, pagar y retirar). Para completarlo se crearon todas las vistas intermedias, con una barra de progreso en cada una.

Tabla 11. *Pasos del proceso de compra*

Paso	Página	Qué hace
1. Carrito	`carrito.html`	Muestra los productos, permite cambiar cantidades y quitar, y ofrece continuar o seguir eligiendo.
2. Entrega	`entrega.html`	Elige retiro en el local o envío a domicilio; pide nombre, teléfono y correo; con envío pide dirección, barrio y franja horaria. Valida los campos y guarda un borrador.
3. Pago	`pago.html`	Elige el medio de pago (Mercado Pago, tarjeta con cuotas, transferencia o efectivo al retirar) y confirma. El cobro es simulado.
4. Confirmación	`confirmacion.html`	Muestra el ticket con número de pedido (por ejemplo AP-01001), el detalle, las instrucciones y permite imprimirlo.

Cada vista tiene guardas: si el carrito está vacío se vuelve al carrito, y no se puede entrar al pago sin haber completado la entrega. «Efectivo al retirar» se deshabilita si el pedido es con envío. Al confirmar, el pedido se guarda en el historial y el carrito se vacía.

9.6 Cuenta de cliente y pedidos

La página de cuenta pasó de tener formularios sin lógica a funcionar en modo simulado. El registro y el inicio de sesión validan los datos, informan errores (por ejemplo, correo ya registrado o credenciales incorrectas) y, con la sesión iniciada, el encabezado muestra «Hola, nombre». El panel de la cuenta lista los pedidos del cliente, permite repetir un pedido con un clic (vuelve a cargar sus productos en el carrito) y cerrar la sesión.

Alcance: al no haber servidor, los usuarios y pedidos se guardan en el navegador. La contraseña nunca se guarda en texto plano, sino como hash SHA-256 con el correo como sal. Esto es solo didáctico: en un sistema real la autenticación debe resolverse en el servidor.

9.7 Eventos utilizados

Se utilizan eventos del navegador y dos eventos personalizados que comunican módulos entre sí.

Tabla 12. *Eventos de JavaScript*

Evento	Dónde se usa	Para qué
`click`	Catálogo, carrito, cuenta, confirmación	Filtros, botones +, − y Quitar, Vaciar, Cerrar sesión, Repetir pedido e Imprimir. Se usa delegación de eventos.
`input`	Buscador del catálogo	Filtra mientras se escribe, con un retardo de 250 ms.
`change`	Orden, cantidad del carrito, formularios de entrega y pago	Reordena, actualiza cantidades y muestra u oculta campos de envío y cuotas.

Evento	Dónde se usa	Para qué
`submit`	Tarjetas de producto, entrega, pago, registro y login	Valida, evita la recarga con `preventDefault` y ejecuta la acción.
`hashchange`	Catálogo	Aplica el filtro cuando cambia el # de la URL.
`storage`	Carrito y sesión	Sincroniza el estado entre pestañas.
`DOMContentLoaded`	`aviso-archivo.js`	Muestra el aviso de servidor local cuando corresponde.
`carrito:cambio` (personalizado)	Carrito, encabezado	Avisa que el carrito cambió para redibujar la vista.
`sesion:cambio` (personalizado)	Cuenta, encabezado	Avisa que se inició o cerró sesión.

9.8 Tipos de funciones utilizados

Tabla 13. *Tipos de funciones y ejemplos*

Tipo	Ejemplo	Archivo
Declaración	`function agregar(codigo, cantidad)`	`modulos/carrito.js`
Flecha	`const buscarPorCodigo = (codigo) => ...`	`modulos/productos.js`
Anónima autoejecutable (IIFE)	`(function () { ... })();`	`aviso-archivo.js`
Callback	`setTimeout(() => ..., 250)`, `.filter(...)`, `.map(...)`, `addEventListener(..., (evento) => ...)`	Vistas y módulos
Asíncrona (`async` / `await`)	`async function hashear(texto)`, `async function iniciar()`	`sesion.js`, `principal.js`
De orden superior	`manejarEnvio(accion, alExito)` devuelve una función	`vistas/cuenta.js`
Función etiqueta	`html` para plantillas con `${...}`	`modulos/plantillas.js`
Importación dinámica	`import('./vistas/catalogo.js')`	`principal.js`

Además se utilizaron una clase (`HtmlSeguro`), desestructuración, operadores `?.` y `??`, parámetros por defecto y métodos de arreglos como `reduce`, `find` y `some`.

9.9 Comentarios y convenciones

Cada archivo comienza con un comentario de cabecera que explica su propósito, y cada función o bloque relevante lleva un comentario que indica qué hace. De las 1.243 líneas de JavaScript, aproximadamente 295 contienen comentarios. Los nombres de variables, funciones y archivos están en español, con camelCase para variables y funciones, y minúsculas para los archivos.

9.10 Verificación

El funcionamiento se verificó sobre un servidor local con Chromium automatizado. Los resultados se resumen a continuación.

Tabla 14. *Verificaciones de la etapa de JavaScript*

Prueba	Alcance	Resultado
Recorrido completo del sitio	46 comprobaciones: catálogo, filtros, búsqueda, carrito, compra con retiro, compra con envío, cuenta, historial, repetir pedido y aviso sin servidor	46 correctas
Accesibilidad automática (axe-core)	8 vistas y estados × 3 anchos (360, 768 y 1280 px)	0 violaciones
Desbordes horizontales	Las mismas 8 vistas y estados × 3 anchos	0 problemas
Sintaxis de los 23 archivos JavaScript	Revisión con Node.js	Sin errores

Limitaciones: las pruebas se hicieron en un solo navegador (Chromium) y sin lector de pantalla. El entorno de prueba no tenía acceso a internet, por lo que no se descargaron las tipografías ni el mapa; esto no afecta al código.

9.11 Cambios respecto de la propuesta

La propuesta original era un sitio estático. Estos son los cambios y decisiones tomados al integrar JavaScript:

Tabla 15. *Cambios respecto de la propuesta*

Propuesta original	Implementado	Motivo
Catálogo escrito en el HTML	Catálogo dibujado desde JavaScript con los datos de un módulo	Requisito de la etapa; evita duplicar datos
Formularios GET hacia `carrito.html`	Eventos `submit` sin recarga y carrito persistente	Un formulario GET no podía agregar productos
Filtros como enlaces a anclas	Botones que filtran de verdad; se agregaron buscador y orden	Mejora de uso; no figuraban en la propuesta
Carrito de ejemplo con 2 productos	Carrito real con cantidades, subtotales y total	Requisito de la etapa
Proceso en tres pasos: elegir, pagar, retirar	Cuatro vistas: carrito, entrega, pago y confirmación	Se agregaron entrega y confirmación para completar el flujo, que ya incluía retiro, envío y número de ticket
Medios de pago solo informativos	Selección del medio, con cuotas para tarjeta; el pago es simulado	Integrar Mercado Pago requiere un servidor y credenciales
«Efectivo al retirar»	Solo disponible cuando el pedido es con retiro	Coherencia con el nombre del medio

Propuesta original	Implementado	Motivo
Cuenta con formularios sin lógica	Registro e inicio de sesión simulados, historial y repetir pedido	Sin servidor, los datos se guardan en el navegador
Recuperar contraseña	No disponible; se informa al usuario	Requiere enviar correos desde un servidor
Costo de envío	No se calcula	La propuesta no define tarifas
Alias de transferencia	Se muestra un alias de ejemplo	La propuesta no incluye el dato real
Apertura del sitio con doble clic	Hay que usar un servidor local; se muestra un aviso si no se hace	Los módulos de JavaScript no funcionan con `file://`

10. Pendientes y recomendaciones

1. **Nomenclatura de clases:** pasar a BEM (`bloque__elemento--modificador`), reemplazar los selectores que dependen de `aria-label` por clases propias y unificar los puntos de corte en tres o cuatro valores.
2. **Datos de ejemplo:** reemplazar el teléfono (0351 400 0000), las URL de Instagram, Facebook y WhatsApp, y el alias de transferencia (`js/datos/pagos.js`) por los reales.
3. **Servidor real:** para vender de verdad hacen falta autenticación con contraseñas protegidas en el servidor, pedidos en una base de datos, integración con Mercado Pago y recálculo del total en el servidor.
4. **Reglas del negocio:** definir costos de envío, control de stock y horarios de retiro.
5. **Archivo sin uso:** `recursos/imagenes/local.svg` ya no se referencia en ninguna página.
6. **Tabla en celular:** al usar Flexbox en las filas, algunos lectores de pantalla pueden dejar de anunciarla como tabla; se resuelve agregando atributos `role`.
7. **Tipografías:** se cargan desde Google Fonts con `@import`; en producción conviene alojarlas localmente o usar `<link>` con precarga.
8. **Más pruebas:** repetir las verificaciones en otros navegadores y dispositivos reales, y con un lector de pantalla.
9. **Entrega:** comprimir el contenido de la carpeta `src` de modo que `index.html` quede en la raíz del archivo entregado.

11. Conclusiones

El proyecto pasó de seis páginas HTML sin estilos y con errores de rutas y formularios a un sitio coherente, responsive y accesible, y luego a una aplicación funcional del lado del cliente: el catálogo se dibuja desde un módulo de datos, el carrito es dinámico y persistente, el proceso de compra tiene todas sus vistas y las cuentas de cliente funcionan en modo simulado.

Cumple los requisitos de estructura, semántica, ARIA, organización de recursos, responsividad e integración con JavaScript; la nomenclatura de clases CSS es el único criterio que queda parcial. El sitio es todavía una demostración sin servidor: el pago es simulado y los datos de clientes y pedidos viven en el navegador. Los pendientes de la sección 10 detallan cómo convertirlo en una tienda real.